Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona
UNIVERSITAT POLITÈCNICA DE CATALUNYA

Iniciació al llenguatge de programació C

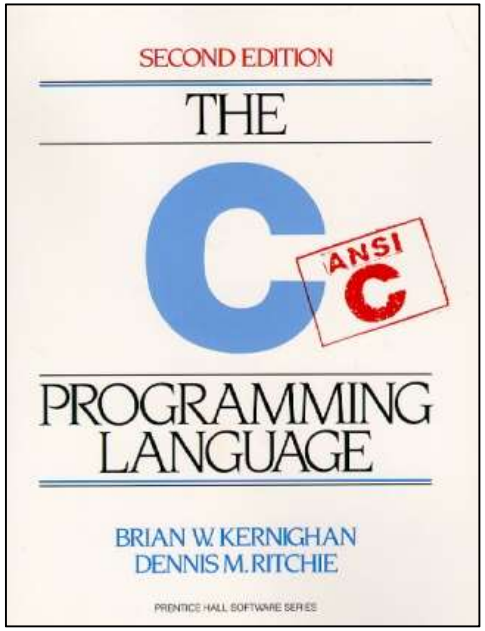
Programació i Estructures de Dades (PRD)

Grau en Enginyeria Electrònica de Telecomunicació

ETSETB Telecom BCN

El llenguatge de programació C (1/4)

- **Creat** als laboratoris Bell (1969-1972) per Dennis M. Ritchie, **per tal d'implementar el sistema operatiu UNIX** . També usat per desenvolupar altres sistemes operatius (Linux, Mac OS), llenguatges de programació (C++, Python, Ruby) i una infinitat d'aplicacions
- Estandarditzat per l'ANSI l'any 1989, donant lloc a la **versió C89** (coneguda com a **C estàndard o ANSI C**), **suportada per la majoria de compiladors**
  - Més endavant: versions C99 (2000), C11 (2011)
- Considerat un **llenguatge de programació de nivell mitjà**, oferint una sintaxi i estructures més properes al raonament humà que al llenguatge màquina, a més d'eines per actuar directament sobre els bits que gestiona l'ordinador i la seva memòria

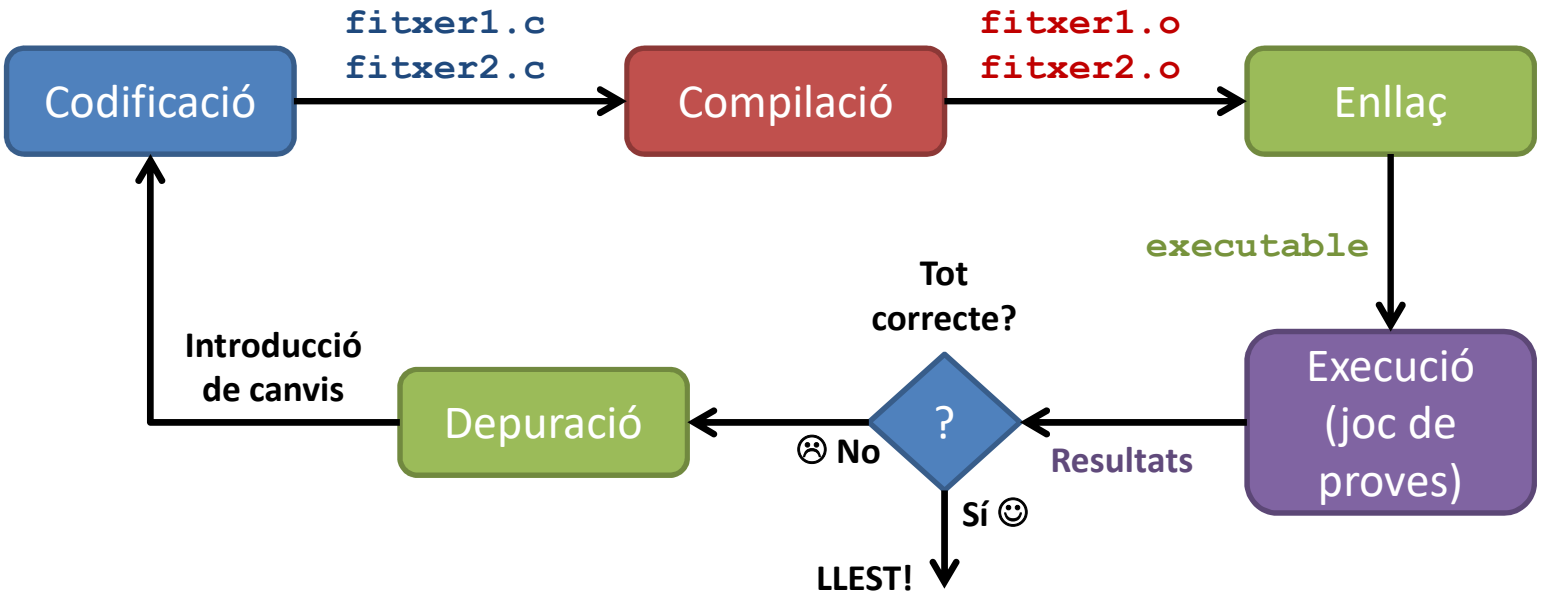


El llenguatge de programació C (2/4)

- A diferència d'altres llenguatges de programació d'alt nivell interpretats (com ara Python), **C és un llenguatge de programació compilat**
- **En comptes de disposar d'un intèrpret** que tradueix durant l'execució del programa les línies de codi font a codi màquina natiu de la plataforma, **a C s'utilitza un compilador**
- El compilador és una eina capaç de detectar errors sintàctics al nostre codi per tal d'acabar generant un **fitxer executable**:
  - Aquest fitxer ja conté el codi màquina natiu de la plataforma, **podent aconseguir així una eficiència superior** (no requereix cap intèrpret)
  - Tanmateix, els llenguatges de programació compilats acostumen a mostrar **certa dependència de la plataforma** on s'acabarà executant l'aplicació (**no ofereixen suport multiplataforma**)

El llenguatge de programació C (3/4)

- Procés de codificació i prova d'un programa en llenguatge C:



- Els **fitxers de codi font C** tenen extensió .c
- El **compilador** tradueix els fitxers de codi font a codi màquina, dipositant-los en **fitxers objecte** amb extensió .o (no executables, de moment)
- **L'enllaçador combina tots els fitxers** .o per acabar **generant un únic fitxer executable** preparat per a ser provat

El llenguatge de programació C (4/4)

- En un entorn Linux podem disposar fàcilment de les eines necessàries per poder codificar, compilar, enllaçar i depurar els nostres programes C. Per exemple:
  - Editor: GNOME gedit
  - Compilador/enllaçador: GNU gcc
  - Depurador: GNU gdb

- Podem utilitzar també un **entorn de desenvolupament integrat (IDE)** que ens facilitarà el desenvolupament dels nostres programes
  - En una mateixa interfície, **l'IDE ens integra un editor per al nostre llenguatge de programació, el compilador/enllaçador i el depurador**
  - En aquesta assignatura utilitzarem **NetBeans* amb extensions per a C/C++** (originari per a Java)

(*) https://netbeans.apache.org

Tipus de dades elementals

- Un programa s'encarrega de processar **dades** (informació) per acabar proporcionant un resultat d'interès:
  - Aquestes dades poden ser **constants**, quan el seu valor no canvia al llarg de l'execució del programa, o **variables**, quan són susceptibles a canviar
  - Són també **d'un tipus determinat** . El llenguatge C considera 3 tipus de dades elementals: enters, reals i caràcters
- A diferència d'altres llenguatges de programació, **C exigeix especificar quin tipus de dada emmagatzemarà una variable** quan es defineix per primera vegada al programa (**quan es declara**)
- S'empren les paraules reservades `int`, `float`, `double` i `char`, que identifiquen els tipus de dada enter, real, real de doble precisió (amb més decimals) i caràcter, respectivament

Declaració i inicialització de variables (1/2)

- Quan es declara una variable, es reserva espai a la memòria del computador per guardar-ne el seu valor:

```
int x;        /* Declaració de la variable entera x */
int i,z,k;    /* Declaració de les variables enteres i,z,k */
char c;       /* Declaració de la variable caràcter c */
float f;      /* Declaració de la variable real f */
double pi;    /* Declaració de la variable real double pi */
```

- Els **noms de les variables** poden contenir lletres, números i _ (guió baix), però no poden començar per un número:
  - El nom **1valor_enter** és erroni, però **valor_enter1** és correcte
- **C és un llenguatge de programació case-sensitive**, que distingeix les MAJÚSCULES i minúscules:
  - valor_enter1 i valor_Enter1 representen noms diferents!

Declaració i inicialització de variables (2/2)

- Un cop declarada la variable, aquesta es pot inicialitzar assignant-li valor:

```
x = 10;
y = 4; z = 9; k = 21;
c = 'A';        /* Els literals caràcter, entre ' ' */
f = 7.5;
pi = 3.141592653589793;
```

- Una variable també es pot declarar i inicialitzar conjuntament, tot a la mateixa línia de codi:

```
int x = 10;
int y = 4, z = 9, k = 21;
char c = 'A';
float f = 7.5;
double pi = 3.141592653589793;
```

Definició de constants

- A la versió C89, les constants es defineixen mitjançant la directiva del preprocessador `#define`. Típicament s'anomenen en MAJÚSCULES, respectant les limitacions existents a l'hora d'anomenar variables:

```
#define NUM_MESOS_ANY 12
#define PI 3.1416
```

- Aquesta directiva indica al compilador que substitueixi totes les aparicions dels textos `NUM_MESOS_ANY` i `PI` al llarg del codi pels textos `12` i `3.1416`. El preprocessador és qui s'encarrega d'aquesta tasca
- En general, **definir constants** simbòliques mitjançant `#define` en comptes d'usar constants literals (valors introduïts directament) **millora la llegibilitat del codi**, sobretot **quan els noms emprats són descriptius**
- **<u>Observació</u>** : les línies de codi que comencen amb `'#'` no acaben en `';'`

Expressions i sentències (1/4)

- Una **expressió** és una **combinació d'operadors, variables i constants** (literals i simbòliques) que, un cop avaluada, **dóna com a resultat un valor**
- Existeixen 3 grups d'operadors a C: aritmètics, relacionals i lògics
- Operadors aritmètics:
  - Suma: +
  - Resta: -
  - Multiplicació: *
  - Divisió (quocient si divisió entera): /
  - Mòdul (residu de divisió entera): %
- Operadors relacionals:
  - Menor que: <
  - Major que: >
  - Igual a : ==
  - Menor o igual que: <=
  - Major o igual que: >=
  - Diferent de: !=

Expressions i sentències (2/4)

- Operadors lògics:
  - And: &&
  - Or : ||
  - Not : !
- Exemples d'expressions amb operadors aritmètics, relacionals i lògics:

`8*2 + 4`	Valor resultant: 20
`7/3`	Valor resultant: 2 (quocient de divisió entera)
`7%3`	Valor resultant: 1 (residu de divisió entera)

```
int i = 2, j = 4;
```

`(i < j) && (i >= j)`	Valor resultant: 0 (fals)
`(i < j) || (i > j)`	Valor resultant: 1 (cert)
`!(i < j)`	Valor resultant: 0 (fals)

Expressions i sentències (3/4)

- Una **sentència** implica una **acció**, com ara l'**assignació del valor d'una expressió a una variable**, o la **crida a una funció**
- Exemples de sentències d'assignació:

```
float base = 2.0;       /* Assigna el valor real 2.0 a base   */
float altura = 4.0;     /* Assigna el valor real 4.0 a altura */
float area = base*altura/2;     /* Assigna el resultat de
                                   l'expressió base*altura/2
                                   a la variable area */
int n;
/* ... */
int es_parell = !(n%2);
```

- **Observacions** : l'assignació s'efectua de dreta cap a esquerra amb l'operador d'assignació = (no confondre'l amb l'operador relacional d'igualtat ==). Totes les sentències finalitzen amb el caràcter ';'

Expressions i sentències (4/4)

- A més, existeixen també **sentències on es crida a una funció** (concepte tractat amb més detall al tema següent)
- Una funció és un **conjunt de sentències**, al qual li assignem un nom, **on es realitzen una sèrie d'operacions** a partir d'unes dades d'entrada**, podent retornar un cert valor**
  - Les funcions poden existir ja a les llibreries de desenvolupament del propi llenguatge (**funcions de llibreria**) **o ser implementades pel programador**, per a l'aplicació concreta
  - Les **llibreries de funcions s'han d'importar prèviament** per poder ser utilitzades (mitjançant la directiva del preprocessador `#include`, seguida del nom de la llibreria entre `< >`)
- En aquest primer tema ens limitarem a usar les funcions `printf()` i `scanf()` disponibles a la llibreria `stdio.h`, que permeten mostrar missatges per pantalla i llegir dades del teclat

La funció printf (1/2)

- En el cas més senzill, la funció `printf()` només requereix que li passem com a paràmetre el missatge que volem mostrar entre " ":

```
# include<stdio.h>
/* ... */
printf("Hola que tal?\n");          /* Invocació de printf() */
printf("Molt be!\n");               /* Invocació de printf() */
```

- Els missatges impresos mitjançant `printf()` finalitzen moltes vegades amb el caràcter d'escapament `'\n'` (salt de línia). Un altre caràcter d'escapament típic és `'\t'` (tabulat)
- Si volem incloure valors de variables al missatge, hem d'usar els especificadors de format `%d`, `%f`, `%lf` o `%c`, depenent de si la variable de la qual se'n vol mostrar el valor és de tipus `int`, `float`, `double` o `char`

La funció printf (2/2)

- Exemples d'ús de `printf()`:

```
# include<stdio.h>
/* ... */
float base = 2.0, altura = 4.0, area;
area = base*altura/2;
printf("El triangle te base %f\n", base);
printf("El triangle te base %f i altura %f\n", base, altura);
printf("L'area del triangle amb base %f i altura %f es %f\n",
base, altura, area);
```

- En comptes de mostrar l'especificador de format, es mostra el valor de la variable passada després del text entre " ". Si es volen incloure els valors de diverses variables, cal introduir els modificadors de format adeqüats al llarg del text i les variables darrere, separades per comes

La funció scanf

- A diferència de `printf()`, la funció `scanf()` permet llegir dades del teclat, habilitant la interacció de l'usuari amb el programa
- En el cas més senzill, només requereix un especificador de format entre " " i la variable on dipositar-hi la dada llegida (precedida per &):

```
#include<stdio.h>
/* ... */
char c; int i; float f; /* Declaració de les variables
                           on s'hi guardaran les dades */
scanf("%c", &c); /* Important & davant la variable */
scanf("%d", &i); /* Important & davant la variable */
scanf("%f", &f); /* Important & davant la variable */
```

Estructura bàsica d'un programa C

```
#include<stdio.h>
#include<stdlib.h>
```

Importació de llibreries

```
#define PERCENTATGE 20
```

Definició de constants simbòliques

```
int main() {
  float valor, res;
  printf ("Introdueix el valor: ");
  scanf ("%f", &valor);
  res = valor*PERCENTATGE/100;
  printf ("El %d%% de %f es %f\n", PERCENTATGE, valor, res);
  return(EXIT_SUCCESS);
}
```

Funció principal (punt inicial d'execució) del programa. TOT PROGRAMA EN C HA D'IMPLEMENTAR AQUESTA FUNCIÓ

printf () i scanf (): Funcions definides a stdio.h

Constant simbòlica definida a stdlib.h -> La funció main() retorna (return) EXIT_SUCCESS per indicar que tot ha anat bé (en cas contrari retornaria EXIT_FAILURE)

Exercicis - I

- **EXERCICI 1.1:** Implementa un programa senzill en C que sol·liciti introduir dos valors enters a través del teclat i imprimeixi per pantalla 1 si el primer >= que el segon, o 0 en cas contrari.
- **EXERCICI 1.2** : Implementa un altre programa senzill en C que actuï com a conversor d'angle en graus -> radians, tenint en compte que 360 graus són 2*pi radians. L'angle en graus cal introduir-lo a través del teclat. Defineix la constant simbòlica PI, el valor de la qual sigui 3.1416.
- **EXERCICI 1.3** : Implementa un programa senzill en C que resolgui l'equació de segon grau $ax^2 + bx + c = 0$ (a != 0), on els coeficients a, b i c caldrà introduir-los a través del teclat.

  Suggerència: podeu usar la funció `sqrt()` definida a la llibreria `math.h` per calcular l'arrel quadrada de b*b – 4*a*c. Aquesta funció requereix com a paràmetre el valor (`float`) del qual cal calcular l'arrel quadrada i retorna el resultat, que pot ser assignat a una variable de tipus `float`.

Sentències de control de flux

- Les sentències de control de flux **permeten controlar el flux d'execució d'un programa en C**. N'existeixen dos grans tipus:
  - **Condicionals**: Permeten vincular l'execució d'un conjunt d'instruccions al compliment d'una condició determinada
  - **Iteratives**: Permeten executar un conjunt d'instruccions un nombre de vegades determinat, o mentre es segueixi complint una certa condició
- Les sentències condicionals poden ser d'alternativa simple (`if`), doble (`if/else` o `if/else` encadenades) i múltiple (`switch/case`)
- Les sentències iteratives poden ser de tipus `while`, `do/while` o `for`
- Les sentències de control de flux són la base de l'algorísmia!

Sentències condicionals if i if/else

- La sentència condicional d'alternativa simple (`if`) s'utilitza per indicar que un conjunt d'**instruccions** s'executaran **només quan** una **condició es compleix**:

```
if (condició certa)
{
     /* Instruccions */
}
```

Condició: expressió que pot contenir operadors aritmètics, relacionals i lògics, que acaba avaluant a cert/fals

- En la sentència condicional d'alternativa doble (if/else) es defineixen **dos grups d'instruccions**, **l'execució dels quals depèn d'una condició concreta**:

```
if (condició certa)
{
     /* Grup d'instruccions 1 */
}
else / *condició falsa */
{
     /* Grup d'instruccions 2 */
}
```

Sentències condicionals if/else encadenades

- Les sentències condicionals anteriors només contemplen una única condició**. En situacions més complexes**, es pot requerir la **comprovació de diverses condicions**, una darrere l'altra, **executant les instruccions associades a la primera que es compleixi**:

```
if (condició 1 certa)
{
    /* Grup d'instruccions 1 */
}
else if (condició 2 certa)
{
    /* Grup d'instruccions 2 */
}
else  /* Cap condició anterior és certa */
{
    /* Grup d'instruccions 3 */
}
```

***No hi ha cap límit pel que fa al nombre de sentències else if encadenades, podent avaluar >2 condicions**

Sentència condicional switch/case

- La sentència condicional d'alternativa múltiple `switch/case` és semblant a les sentències `if/else` encadenades. En aquest cas, les condicions que cal comprovar sempre són d'igualtat (`expressió == valor`):

```
switch(expressió)
{
 case valor1: /* expressió == valor1 */
      /* Grup d'instruccions 1 */
      break;
 case valor2: /* expressió == valor2 */
      /* Grup d'instruccions 2 */
      break;
 case valor3: /* expressió == valor3 */
      /* Grup d'instruccions 3 */
      break ;
 default : /* Valor != anteriors */
      /* Grup d'instruccions 4 */
}
```

Opcional

***Els valors associats a cada cas només poden ser constants de tipus enter o caràcter**

***El break es requereix al final de cada cas perquè no s'executin les instruccions dels casos següents**

***No existeix límit en el nombre de casos, podent avaluar-se >3 valors específics**

Exemples d'ús de sentències condicionals (1/2)

```
float base, altura, area;
printf ("Introdueix base i
   altura del triangle separades
   per espai: ");
scanf(%f %f, &base, &altura);
if(base>=0.0 && altura>=0.0)
{
    area = base*altura/2;
    printf("area = %f\n", area);
}
else
{
  printf ("base i altura haurien
           de ser >= 0!\n");
}
```

Exemple amb if/else

```
float nota;
scanf("%f", &nota);
if(nota >= 0.0 && nota < 5.0)
    printf ("suspens\n");
else if(nota >= 5.0 && nota < 7.0)
    printf ("aprovat\n");
else if(nota >= 7.0 && nota < 9.0)
    printf ("notable\n");
else if(nota >= 9.0 && nota <10.0)
    printf ("excel·lent\n");
else if(nota == 10.0)
    printf ("MH!\n");
else
    printf ("Nota invàlida\n");
```

Exemple amb if/else encadenades

Exemples d'ús de sentències condicionals (2/2)

```
int dia;
printf ("Introdueix el numero
      de dia de la setmana: ");
scanf("%d", &dia);
switch(dia)
{
  case 1:
    printf("Es dilluns\n");
    break;
  case 2:
    printf("Es dimarts\n");
    break;
  //..
  case 7:
    printf("Es diumenge\n");
    break;
  default :
    printf("Dia invalid!");
}
```

Exemple amb switch /case - 1

```
int dia;
scanf("%d",&dia);
switch(dia)
{
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
    case 6:
      printf("Es laborable\n");
      break;
    case 7:
      printf("Es festiu\n");
      break;
    default:
      printf("Dia invalid!");
}
```

Exemple amb switch /case - 2

Sentències iteratives while i do/while

- La sentència iterativa `while` la usarem per introduir bucles al nostre programa C, que continguin un grup d'instruccions que s'executaran de forma repetida, fins que una certa condició deixi de complir-se:

```
while(condició certa)
{
    /* Instruccions */
}
```

Evidentment, les instruccions han d'assegurar que la condició deixi de complir-se en algun moment...

- Primer es comprova que la condició sigui certa. En cas afirmatiu, s'executen les instruccions. En canvi, a la sentència iterativa `do/while`, primer s'executen les instruccions i després es comprova si la condició segueix complint-se, per tal procedir a efectuar una altra iteració addicional:

```
do
{
    /* Instruccions */
}
while(condició certa);
```

Compte amb la sentència iterativa do/ while: acaba amb ;

Sentència iterativa for

- A diferència de `while` i `do/while`, la sentència iterativa `for` s'utilitza en aquells casos en què el nombre d'iteracions a realitzar es coneix a priori:
  - Es defineix una variable índex que s'inicialitza a l'inici del bucle i s'actualitza al final de cada iteració
  - El bucle `for` continua executant les instruccions que conté, mentre es compleixi una certa condició de control
  - El valor inicial de la variable índex, el criteri d'actualització i la condició de control defineixen el nombre de vegades que es repetirà el bucle

```
/* Declaració de la variable índex, anomenada var */
for(var = valor inicial; cond. control; criteri actualització)
{
    /* Instruccions */
}
```

Exemples d'ús de sentències iteratives

```
int num ;
printf("Dona'm un nombre parell: ");
scanf("%d", &num);
while(num%2 != 0)
{
  printf("No es parell! Torna-hi: ");
  scanf ("%d", &num);
}
printf ("Gracies! Ara dona'm un
         nombre imparell: ");
do
{
  scanf ("%d", &num);
  if (num%2 == 0)
     printf ("Torna-hi: ");
}
while(num%2 == 0);
printf ("Gràcies altra vegada!\n");
```

Exemple amb while i do/ while

```
int min_val = 100;
int max_val = 212;
printf ("Imprimint tots els
      numeros entre [%d, %d]\n",
      min_val,max_val);

int i;
for(i=min_val; i<=max_val; i++)
{
    printf ("\t%d\n", i);
}
```

Exemple amb for

Exercicis - II

- **EXERCICI 1.4** : Implementa un programa C on s'inicialitzi una variable entera amb un valor entre [0,99], que assumirem desconegut per a l'usuari del programa. El programa demanarà d'introduir un valor i comunicarà si el valor introduït és menor, major o igual al valor desconegut. Aquest procés es repetirà fins a esbrinar el valor desconegut.
- **EXERCICI 1.5:** Implementa un programa C que calculi el factorial d'un valor enter positiu. El programa demanarà aquest valor a l'usuari i en mostrarà el resultat per pantalla (valor! = resultat).
- **EXERCICI 1.6** : Implementa un programa en C que mostri els 15 primers valors de la successió de Fibonacci. Aquesta és una successió infinita de números, on cada número és resultat de sumar els seus dos números immediatament anteriors, començant per 0 i 1 (0,1,1,2,3,5,8,13,21, 34,...)

Tipus de dades estructurats

- Els tipus de dades estructurats són **agrupacions d'altres tipus de dades seguint una certa estructura** que permet emmagatzemar **informació més complexa que els tipus de dades elementals**
  - El seu principal objectiu és facilitar la gestió d'aquesta informació més complexa al programa
- Els tipus de dades estructurats més comuns a C són:
  - **Vectors** (també anomenats *arrays*)
  - **Cadenes de caràcters** (també anomenades *strings*)
  - **Estructures** (també anomenades *structs*)
- Cadascun d'aquests tipus presenta unes característiques diferents que n'aconsellaran (o no) el seu ús en cada situació específica

Vectors

- Els vectors en C són estructures que permeten emmagatzemar un **conjunt de variables, totes del mateix tipus, en posicions contigües de memòria**:
  - **Les variables queden identificades per la posició (índex) que ocupen al vector**, per tal de poder-hi accedir posteriorment
  - Les posicions d'un vector **comencen a identificar-se des de l'índex 0 fins a l'índex nombre de posicions – 1**
  - A l'estar emmagatzemat en posicions contigües de memòria, és fàcil accedir a la posició i-èsima d'un vector (sabent quan ocupa cadascuna de les seves posicions en memòria)
- Exemple de vector de 10 posicions, de variables de tipus `int`:

4	22	6	-12	88	32	11	-4	19	8
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

← **Posicions**

Declaració i inicialització de vectors (1/2)

- Els vectors són variables del programa: cal declarar-los abans de ser usats, especificant el tipus de variables que guarda, el seu nom i la seva mida. Per exemple:

```
int valors[10];            /* Vector d'enters de 10 posicions  */
float mesures[500];        /* Vector de reals de 500 posicions */
double mesures_precises[500]; /* De reals de doble precisió */
```

- Un cop declarat, podem assignar valor a les diferents posicions del vector, element a element:

```
valors[0] = 4;
valors[1] = 6;
scanf("%d\n", &valors[2]); /* Demanem valor i emmagatzemem */
/* ... */
```

Declaració i inicialització de vectors (2/2)

- Alternativament, també podem inicialitzar totes les posicions del vector a la vegada, assignant al vector una llista de valors separats per comes, tants com posicions tingui el vector:

```
int valors[10] = {1,2,3,4,5,6,7,8,9,10};
```

- Un cop emplenat el vector, podem obtenir els valors emmagatzemats a les seves posicions a partir del seu índex, p.ex., mitjançant un bucle for:

```
int i;
float valor_total = 0.0, mitjana;
for(i = 0; i < 10; i++){
   valor_total = valor_total + valors[i];
   printf("valors[%d] = %d\n", i, valors[i]);
}

mitjana = valor_total/10;
printf ("\nValor mitjana: %f\n", mitjana);
```

Operacions amb vectors

- No podem realitzar operacions amb vectors directament (p.e., sumar, restar, multiplicar, etc..., vectors)
- Les operacions amb vectors s'han d'efectuar element a element:

```
/* Suma de dos vectors d'enters de 10 posicions */
int vector1[10] = {1,2,3,4,5,6,7,8,9,10};
int vector2[10] = {3,4,5,6,7,8,9,10,11,12};
int vector_suma[10];
int i;
for(i=0; i < 10; i++) {
   vector_suma[i] = vector1[i] + vector2[i];
}
```

Matrius (1/2)

- Les matrius són **vectors bidimensionals formats per files i columnes**, quedant **cada posició identificada per la fila i columna que representa**
- Les matrius en C **s'utilitzen de forma exactament igual als vectors** . Però requerirem dos índexs per referir-nos a cadascuna de les seves posicions
- A més, en el moment de declarar una matriu, haurem d'especificar el seu nombre de files i columnes:

```
int matriu_enters[5][6];   /* Matriu de 5 files i 6 columnes */

/* Inicialitzem totes les seves posicions a 0 */
int i, j;
for(i=0; i < 5; i++) {
   for(j=0; j < 6; j++) {
      matriu_enters[i][j] = 0;
   }
}
```

Matrius (2/2)

- Seguint l'exemple de la transparència anterior...

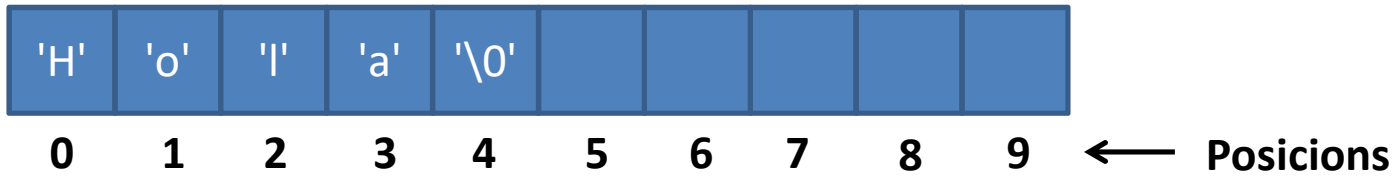
```
/* Després d'assignar valors a les posicions de la matriu,
podríem mostrar els elements de la primera fila */
for(j=0; j<6; j++)
{
   printf ("matriu_enters[0][%d] = %d\n", j,
                                        matriu_enters[0][j]);
}

/* I els de la primera columna? */
for(i=0; i<5; i++)
{
   printf ("matriu_enters[%d][0] = %d\n", i,
                                        matriu_enters[i][0]);
}
```

Exercicis - III

- **EXERCICI 1.7:** Implementa un programa C on es declari i s'inicialitzi un vector d'enters. El programa ha de mostrar per pantalla el mòdul d'aquest vector, és a dir, l'arrel quadrada de la suma dels quadrats dels valors guardats a cadascuna de les seves posicions.
- **EXERCICI 1.8** : Implementa un programa C on es declari i s'inicialitzi un vector d'enters de 10 posicions. Tot seguit, el programa ha d'invertir aquest vector, intercanviant els valors de la primera i última posició, segona i penúltima posició etc..., sense fer servir cap altre vector auxiliar. El programa ha de mostrar per pantalla el contingut del vector abans i després de la inversió.
- **EXERCICI 1.9** : Implementa un programa C on es declari i s'inicialitzi una matriu de valors reals amb mateix nombre de files i columnes. El programa ha de detectar i informar de si la matriu presenta simetria respecte a la diagonal principal o no.

Cadenes de caràcters

- Les cadenes de caràcters (*strings*) **ens permeten guardar text al nostre programa C**. Essencialment, **són vectors unidimensionals que guarden variables de tipus `char` a les seves posicions**
- Tot i això, presenten algunes particularitats:
  - Inclouen un caràcter especial per indicar el final de la cadena, tot i quedar encara posicions lliures al vector ('\0')
  - El llenguatge C inclou funcions de llibreria per operar còmodament amb cadenes de caràcters (definides a la llibreria `string.h`)
- Exemple de cadena de caràcters emmagatzemada en un vector de variables de tipus `char` de 10 posicions:



Cadenes de caràcters

- Les cadenes de caràcters es poden inicialitzar de diferents maneres. Podem assignar al vector de variables de tipus `char` declarat, una constant literal de tipus text entre "...":

```
char paraula[5] = "Hola "; /* Afegeix automàticament '\0'
                              al final; per això mida = 5 */
```

- Una altra opció és inicialitzar-la posició a posició:

```
char paraula[5];
paraula[0] = 'H';
paraula[1] = 'o';
paraula[2] = 'l';
paraula[3] = 'a';
paraula[4] = '\0'; /* Aquí cal afegir-lo manualment */
```

- A partir d'aquest moment, podrem accedir als diferents caràcters de la cadena tal i com accediríem a les posicions de qualsevol vector

Funcions de llibreria string.h

- El llenguatge de programació C proporciona diverses funcions de llibreria per facilitar el treball amb cadenes de caràcters, disponibles a string.h
- Per exemple:

```
strlen(cadena); /* Mida cadena (num . chars) abans del '\0' */

strcpy(cadena_dest, cadena_orig); /* Còpia contingut de
cadena_orig (abans de '\0') a cadena_dest . cadena_dest ha de
ser suficientment gran! */

strcat(cadena_dest, cadena); /* Concatena cadena al final de
cadena_dest. cadena_dest ha de ser suficientment gran! */

strcmp(cadena_1, cadena_2); /* Retorna >0 si cadena_1 >
cadena_2, <0 si cadena_1 < cadena_2, o 0 si ambdues són iguals,
realitzant una comparació lexicogràfica, caràcter a caràcter */
```

Exercicis - IV

- **<u>EXERCICI 1.10 :</u>** Implementa un programa C on s'introdueixin dues cadenes de caràcters a través del teclat. El programa ha de mostrar per pantalla quina de totes dues és més llarga en caràcters, o si totes dues tenen igual longitud. Tingueu en compte el codi d'exemple següent per a la lectura d'una cadena de caràcters del teclat i la seva visualització posterior:

```
char cadena[40];
fgets(cadena, 40, stdin);
printf("%s", cadena);
```

- **<u>EXERCICI 1.11:</u>** Implementa un programa C que permeti introduir una cadena de caràcters, una paraula, a través del teclat. El programa ha de ser capaç de lletrejar la paraula introduïda, mostrant-la per pantalla. Per exemple, si la paraula introduïda és "Hola", hauria de mostrar "H-o-l-a".

Structs (1/3)

- Els structs són **estructures de dades que permeten desar un conjunt de variables**. Aquestes variables poden ser **de diferents tipus**, a diferència dels vectors
  - Cadascuna de les variables dins d'un struct es denomina camp
  - L'accés a les variables dins d'un struct s'efectua a partir del nom del camp, no pas de la seva posició
- Per afavorir la llegibilitat del codi, els structs es defineixen al principi del fitxer de codi font (després de les directives `#include` i `#define`)

```
#include<stdio.h>
#include<stdlib.h>
struct complex_number{
   float real;
   float imaginary;
};
/* ... */
```

Ull: la definició d'un struct acaba en ;

Structs (2/3)

- Un cop definit l'struct, podem començar a declarar variables d'aquest tipus. L'accés als seus camps es realitza mitjançant l'operador "."

```
/* Seguint amb el codi anterior... */
int main() {
    struct complex_number c1, c2, suma;
    c1.real = 1.0; c1.imaginary = 1.0;
    c2.real = 2.0; c2.imaginary = 2.0;
    suma.real = c1.real + c2.real;
    suma.imaginary = c1.imaginary + c2.imaginary;
    printf("Suma: %f + %fi\n", suma.real, suma.imaginary);
    return(EXIT_SUCESS);
}
```

- Les operacions amb structs s'han d'efectuar camp a camp, p.e ., no podent comparar-los directament (sí que es poden però copiar directament: struct complex_number c3 = c2;)

Structs (3/3)

- La definició d'un struct comporta molts cops l'ús de l'operador `typedef`, que permet definir un sinònim per a un cert tipus de dada. Tornant a l'exemple anterior:

```
struct complex_number{
   float real;
   float imaginary;
};
typedef struct complex_number t_complex;

int main() {
    t_complex c1, c2, suma;
    /* ... */
}
```

Sinònim

També podem definir struct i sinònim conjuntament

```
typedef struct {
   float real;
   float imaginary;
}t_complex;
```

Un cop utilitzat typedef, quan vulguem declarar una variable de tipus struct complex_number podem emprar el seu sinònim t_complex

Exercicis - V

- **<u>EXERCICI 1.12:</u>** Definir un nou struct `t_nif` compost per dos camps: un `int` anomenat número i un `char` anomenat lletra. Implementar un programa C capaç de detectar si dues variables de tipus `t_nif` representen el mateix NIF o no.
- **<u>EXERCICI 1.13</u>** : Definir un nou struct `t_alumne` compost per un camp de tipus `t_nif` que guardi el NIF de l'alumne i 3 camps de tipus `float` que guardin les notes d'examen parcial, laboratori i final. Implementar un programa C capaç d'obtenir la nota final d'un alumne aplicant la fórmula d'avaluació de l'assignatura.
- **<u>EXERCICI 1.14</u>** : Definir un nou struct `t_grup`, compost per dos camps de tipus `int` anomenats grup i num_alumnes, a més d'un camp de tipus vector de variables de tipus `t_alumne` anomenat matriculats, de mida igual a la constant `MAX_ALUMNES`. Implementar un programa C capaç de mostrar per pantalla els NIFs dels alumnes matriculats a un cert grup.